



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Kairix

加速文档

参赛队名 Unicus

队伍成员 颜晨、雷鑫言、萧鹏

指导老师 夏文、仇洁婷

2026 年 8 月 12 日

目录

摘要	3
1. 问题 (bug 或性能瓶颈) 定位与根因分析	3
1.1: 瓶颈 1: 全局堆锁令多核编译退化为内核自旋	3
1.2 瓶颈 2: refill/drain 仍持有全局锁做批量工作	4
2. 修复或优化的设计与实现	6
2.1 per-CPU magazine (1c0d99aa)	6
2.2 per-CPU slab (b7fb26d6)	8
3. 实验分析: 修改前后的编译时间、加速比等数据对比	11
3.1 修改前后的编译时间:	11
3.2 加速比	12
4. AI 使用说明与可复现步骤	13
4.1 AI 使用说明	13
4.2 可复现步骤	13
4.2.1 实验环境	13
4.2.2 构建与运行	14
4.2.3 本地诊断脚本说明	15
4.2.4 使用官方评测流程交叉验证	15

摘要

BuildStorm 性能瓶颈主要位于 Kairix 的通用内存管理路径，主要是内核 heap 模块。这些代码由 RISC-V 与 LoongArch 共享，相关优化也在两个架构中同步实现。因此，本文以 RISC-V 日志进行定量分析；共享代码上的瓶颈机理和优化方案同样适用于 LoongArch

本文主要分析 RISC-V 同一测试环境下 buildstorm_testcode.sh 在三个提交点上的日志：

版本	提交	日志状态	主要结果
A (经过所有优化后)	b50a290f9e34d670e14191582d13740ff92f0016	完成	总耗时 1378.06s, 主编译 1267.53s
B (堆缓存优化第一版)	1c0d99aab415ba866cbe9852b9493b70f73cb40f	完成	总耗时 2696.44s, 主编译 2584.38s
C (基线)	7961ad6f2dbe5ff2eb87dff1de0e3ab80e370e08	未完成	已运行至少 14818.70s, 仍停留在主编译

核心结论如下。

1.C 编译速度慢的原因并不是因为 Rust 编译器计算量过大，也不是互斥锁意义上的“死锁”，而是全局内核堆锁导致的自旋竞争。

2.B 加入 per-CPU 小对象 magazine 后缓解了全局锁的竞争，但只是缓解，可以看到 897109 次全局堆锁获取中有 197721 次竞争，累计等待 4204.80s，最长一次 1.355s。77.86% 的竞争来自 magazine 的 refill/drain；它降低了进锁频率，却仍在全局锁内逐个分配或释放 16 个对象，临界区仍然过长。

3.A 进一步引入真正的 per-CPU slab、匿名页批量分配。A 与 B 的用户态 CPU 时间只差 0.73%，但 B 的内核态 CPU 时间是 A 的 5.18 倍。A 的主编译相对 B 加速 2.04 倍，总耗时下降 48.89%。

4.因为 C 没有完成，因此说加速比下界：B 相对 C 的总流程加速比大于 5.50 倍，A 相对 C 大于 10.75 倍。

1. 问题 (bug 或性能瓶颈) 定位与根因分析

1.1: 瓶颈 1：全局堆锁令多核编译退化为内核自旋

在分析 C 的日志的时候，可以清晰地看到 C 的`arceos\_build`部分累计 CPU 构成随着时间加长而恶化：

主编译累计时间	用户态占比	内核态占比	用户态等效核数	内核态等效核数
947.52s	9.98%	46.95%	0.80	3.76
1182.45s	8.60%	56.86%	0.69	4.55
4036.09s	3.85%	84.40%	0.31	6.75
14707.60s	3.65%	88.55%	0.29	7.08

我刚开始怀疑的瓶颈是 rustc 本身又或者是 IO 交互的问题，因为没有死锁，然后又要等待很久，因此最初的怀疑是这两个方向。分析了日志之后，我感觉不太对劲，因为假如瓶颈是 rustc 本身，CPU 时间应主要计入用户态，那么日志表现出来的现象应该是用户态占比较大；如果瓶颈是磁盘等待，应主要表现为空闲率升高。可以看到日志的现象恰好和这两个假设相反：内核态占满绝大部分 CPU，而真正执行编译器的用户态份额持续被挤压。所以这两个地方都被我排除了，又因为内核态占比越来越大，所以我开始怀疑是不是多核在某个地方出现了严重的锁竞争或忙等。

分析代码之后，我可以找到与现象能够闭环的代码：

在 os/src/mm/heap_allocator.rs 中：

```
1 HEAP_ALLOCATOR.inner: SpinNoIrqLock<Heap>
```

我们的堆的分配和释放都采用的是关中断锁，每个 CPU 在进入该锁的获取/持有路径时关闭自己的本地中断，但所有 CPU 争抢的是同一个全局 heap 锁对象。比如 alloc 在每一次调用中都要执行 self.lock(HEAP_OP_ALLOC,...)。

```
1 unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
2     let mut heap =
3         self.lock(
4             HEAP_OP_ALLOC,
5             0,
6             layout.size(),
7             layout.align(),
8         );
9
10    heap.alloc(layout)
11        .map_or(core::ptr::null_mut(), |allocation| {
12            allocation.as_ptr()
13        })
14 }
```

又因为并行 rustc 会通过 VFS、page cache、dentry、进程和内存映射产生大量短生命周期内核对象，进而导致 8 个 CPU 不断争抢同一个关闭本地中断的自旋锁，又因为等待者不是睡眠，而是在内核态轮询锁，这些因素最终形成接近卡死的现象：

对象并发分配/释放->单一 buddy heap 锁->多核自旋->内核态 CPU 接近 7 核->rustc 用户态只剩约 0.3 核->编译缓慢前进。

1.2 瓶颈 2: refill/drain 仍持有全局锁做批量工作

我们在`1c0d99aa`这个提交为 16B 至 4096B 的 9 个 size class 增加 per-CPU magazine：每类最多缓存 64 个对象，空时 refill16 个，满时 drain16 个，普通分配命中只需要访问当前 CPU 的本地链表，不再访问全局 buddy heap，因此 B 能在 2696.44s 内完成。

但是 2696s 已经接近评测机的 50 分钟超时，因此还需要优化

通过分析日志可以发现全局锁仍是剩余瓶颈：

B 的最终 heap 指标	数值	解释
heap_cache_hits	34452583	per-CPU 缓存命中
heap_cache_misses	241836	缓存未命中
缓存命中率	99.303%	hits / (hits + misses)
heap_lock_acquisitions	897109	全局堆锁获取次数
heap_lock_contended	197721	首次尝试失败, 竞争率 22.04%
heap_lock_wait_ns	4204795097760 ns	所有 CPU 累计等待 4204.80s
平均每次竞争等待	21.27 ms	wait/contended
heap_lock_max_wait_ns	1355220240 ns	最长 1.355 s
refill 竞争	70700	占全部竞争 35.76%
drain 竞争	83251	占全部竞争 42.10%

通过日志可以发现在整个主编译阶段, 平均只有约 1.44 个 CPU 执行 rustc 用户态代码, 却有约 3.58 个 CPU 消耗在内核态。如果剩余瓶颈主要来自 rustc 自身的代码生成, CPU 时间应主要计入用户态; 这里内核态 CPU 时间是用户态的约 2.49 倍, 说明优化后的主要瓶颈仍然位于内核并发路径。

结合 heap 性能计数, 可以发现问题还在全局 buddy heap 锁。为什么优化了一次堆的缓存但是还没有解决根本原因? 为了解决这个问题, 我细化了日志, 进一步统计了 refill 和 drain 的竞争, 想要找到剩下的锁抢占是在哪里? 也就是日志中的:

```
1 HEAP_OP_ALLOC
2 HEAP_OP_DEALLOC
3 HEAP_OP_CACHE_REFILL
4 HEAP_OP_CACHE_DRAIN
```

这样可以区分:

1. 大对象或非缓存对象的直接分配;
2. 对象直接释放到全局 heap;
3. per-CPU magazine 的 refill;
4. per-CPU magazine 的 drain。

在日志中, 可以看到 refill 与 drain 合计占全部竞争的 77.86%。因此, 剩余全局 heap 锁竞争并不主要来自大对象的直接分配, 而是集中在 per-CPU magazine 的 refill 和 drain 慢路径。

我分析了一下多核下堆的释放和分配, 发现根本原因在于 alloc_from_cpu_cache 获取全局锁后, 循环调用 buddy alloc 最多 16 次; dealloc_to_cpu_cache 也在一次全局锁临界区内释放最多 16 个对象。虽然这些 CPU 操作的是各自的本地 magazine, 但慢路径最终都进入同一个全局 buddy heap:

```
1 let mut heap = allocator.lock(
```

```

2     HEAP_OP_CACHE_REFILL,
3     0,
4     cache_layout.size(),
5     cache_layout.align(),
6 );
7
8 let mut count = 0;
9 while count < HEAP_CACHE_REFILL_BATCH {
10     match heap.alloc(cache_layout) {
11         Ok(allocation) => {
12             blocks[count] = allocation.as_ptr() as usize;
13             count += 1;
14         }
15         Err(()) => break,
16     }
17 }

```

小对象很多的时候，即使命中率达到 99.3%，但是剩余 0.7% 的 miss 和频繁 drain 足足有二十多万次，仍足以制造长临界区和锁队列。当一个 CPU 持锁执行最多 16 次 buddy 操作时，其他 CPU 会在内核态自旋等待。随着并发 refill/drain 增加，等待队列增长，真正执行 rustc 用户态代码的 CPU 时间再次被压缩。

2. 修复或优化的设计与实现

2.1 per-CPU magazine (1c0d99aa)

我第一次优化的设计目标是让小对象命中走 CPU 本地路径，这样可以将全局 buddy 锁从“每对象一次”降为“每批一次”，降低全局锁带来的竞争。实现要点：

1. 将 16B ~ 4KiB 按 2 的幂划为 9 类；
2. 每个 CPU、每个 size class 保存独立空闲链；
3. 本地链空时从全局 heap 一次补充 16 个对象；
4. 本地链达到 64 个时一次归还 16 个对象；

这样设计的思路主要是考虑到 buddy allocator 本身就是按 2 的幂管理内存块，同时 Rust 里面内存请求经常要满足对齐要求。因此这样分为九类基本上可以覆盖不同的对象大小。并且幂运算的运算成本是 $O(1)$ 。

而每个 CPU、每个 size class 保存独立空闲链，主要是考虑到要将全局的锁拆分到每个 CPU 里面，同时链表是一个查询成本很低的数据结构，因此就设计了每个 CPU、每个 size 都涉及一个链表来保存指针。

```

1 const HEAP_CACHE_MIN_SIZE: usize = 16;
2 const HEAP_CACHE_MAX_SIZE: usize = PAGE_SIZE;
3 const HEAP_CACHE_CLASS_COUNT: usize = 9;
4 const HEAP_CACHE_CAPACITY: usize = 64;
5 const HEAP_CACHE_REFILL_BATCH: usize = 16;
6 const HEAP_CACHE_DRAIN_BATCH: usize = 16;
7

```

```

8 struct HeapCacheBin {
9     head: usize,
10    len: usize,
11 }
12
13 struct PerCpuHeapCache {
14     bins: [HeapCacheBin; HEAP_CACHE_CLASS_COUNT],
15 }
16
17 static HEAP_CPU_CACHES: [SpinNoIrqLock<PerCpuHeapCache>; MAX_CPU_NUM]
    =
18     [const { SpinNoIrqLock::new(PerCpuHeapCache::new()) };
    MAX_CPU_NUM];
19
20 unsafe impl GlobalAlloc for KernelHeapAllocator {
21     unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
22         if let Some(class) = heap_cache_class(layout) {
23             unsafe { alloc_from_cpu_cache(self, layout, class) }
24         } else {
25             alloc_from_global_heap(self, layout, HEAP_OP_ALLOC)
26         }
27     }
28     // dealloc 同样先判断 size class, 小对象进入本地 cache。
29 }

```

而把全局 heap 操作批量化这一步, 说实话, 当时认为是优化, 因为觉得批量操作可以节约一点调度成本, 可是现在写文档的时候才后知后觉, 在八核疯狂争抢锁的压力下, 是否批量处理其实对结果不会产生太大的优化。下面的实际代码也直接解释了为什么 B 仍慢

```

1 // refill: 本地链为空后进入全局 heap。
2 let mut blocks = [0usize; HEAP_CACHE_REFILL_BATCH];
3 let mut heap = allocator.lock
4     HEAP_OP_CACHE_REFILL,
5     0,
6     cache_layout.size(),
7     cache_layout.align(),
8 );
9 let mut count = 0;
10 while count < blocks.len() {
11     match heap.alloc cache_layout) {
12         Ok allocation) => {
13             blocks[count] = allocation.as_ptr() as usize;
14             count += 1;
15         }
16         Err(()) => break,
17     }

```

```

18 }
19
20 // drain: 本地链过满后，在同一个全局锁内逐个归还。
21 let mut heap = allocator.lock
22     HEAP_OP_CACHE_DRAIN,
23     drained[0],
24     cache_layout.size(),
25     cache_layout.align(),
26 );
27 for pointer in drained[..drain_count].iter().copied() {
28     unsafe {
29         heap.dealloc
30             NonNull::new_unchecked pointer as *mut u8),
31             cache_layout,
32     );
33     }
34 }

```

因此，magazine 把“每个小对象进一次全局锁”改成了“每 16 个对象进一次全局锁”，但没有改变全局 buddy 仍逐对象处理 split/merge 的事实。这部分也就是后续的优化需要解决的问题。

2.2 per-CPU slab (b7fb26d6)

考虑到瓶颈是全局堆锁，我的想法是将分配对象的权力下放到每一个 CPU，所以关键便是 span 的产生，因此我的第二次优化便是将 magazine 改为 slab span：

1. 全局 buddy heap 只提供或回收完整的 64KiB span；
2. 每个 CPU 维护 9 个 slab class；
3. span 内用 bitmap 管理小对象，不再让全局 buddy 逐对象 split/merge；
4. buddy heap 的每阶空闲结构从线性链表改为 AVL tree，使 buddy 查找/删除从线性扫描降为对数复杂度。

主要层次结构

```

1 全局 buddy
2  |
3  | 偶尔分配/回收完整的 64 KiB span
4  ↓
5 CPU 0 slab    CPU 1 slab    CPU 2 slab
6  |            |            |
7 多个 span     多个 span     多个 span
8  |
9 位图管理 slot

```

首先便是，全局的 buddy 只分配回收完整的 span，每个 span 有固定 owner、size class、空闲计数和 bitmap；通过地址向下对齐即可从任意对象找到所属 span。

```

1 const HEAP_SLAB_MIN_SIZE: usize = 16;

```



```

2 const HEAP_SLAB_MAX_SIZE: usize = PAGE_SIZE;
3 const HEAP_SLAB_CLASS_COUNT: usize = 9;
4 const HEAP_SLAB_SPAN_SIZE: usize = 64 * 1024;
5 const HEAP_SLAB_EMPTY_RESERVE: usize = 1;
6
7 #[repr(C, align(64))]
8 struct HeapSlabSpan {
9     magic: usize,
10    owner_cpu: usize,
11    class: usize,
12    next: usize,
13    slot_start: usize,
14    slot_count: usize,
15    free_count: usize,
16    bitmap_hint: usize,
17    free_bitmap: [usize; HEAP_SLAB_BITMAP_WORDS],
18 }
19
20 #[repr(align(64))]
21 struct PerCpuHeapCacheCell {
22     inner: SpinNoIrqLock<PerCpuHeapCache>,
23 }
24
25 static HEAP_CPU_CACHES: [PerCpuHeapCacheCell; MAX_CPU_NUM] =
26     [const { PerCpuHeapCacheCell::new() }; MAX_CPU_NUM];
27
28 #[inline]
29 fn heap_slab_span_base(pointer: usize) -> usize {
30     pointer & !(HEAP_SLAB_SPAN_SIZE - 1)
31 }

```

这样就能解决 refill 和 drain 的问题了，新的 refill 不再持有全局锁循环分配 16 个小块，而是只向全局 buddy 请求一个完整 span，之后所有 slot 切分都在 per-CPU slab 内完成，同理 drain 也是：

```

1 unsafe fn alloc_from_cpu_slab(
2     allocator: &KernelHeapAllocator,
3     layout: Layout,
4     class: usize,
5 ) -> *mut u8 {
6     let cpu = heap_cpu_index();
7     let cached = {
8         let mut cache = HEAP_CPU_CACHES[cpu].inner.lock();
9         unsafe { heap_slab_take_from_class(&mut cache.classes[class]) }
10    };

```

```

11     if let Some(pointer) = cached {
12         return pointer as *mut u8;
13     }
14
15     // 全局 heap 只看见一个 64 KiB allocation。
16     let span_layout = heap_slab_span_layout();
17     let span_address =
18         alloc_from_global_heap(allocator, span_layout,
19     HEAP_OP_CACHE_REFILL) as usize;
19     if span_address == 0 {
20         return core::ptr::null_mut();
21     }
22
23     let refill_cpu = heap_cpu_index();
24     unsafe { initialize_heap_slab_span(span_address, refill_cpu,
25     class) };
25     let pointer = {
26         let mut cache = HEAP_CPU_CACHES[refill_cpu].inner.lock();
27         unsafe {
28             heap_slab_insert(&mut cache.classes[class], span_address);
29             heap_slab_take_from_class(&mut cache.classes[class])
30                 .expect("new slab span must contain a slot")
31         }
32     };
33     pointer as *mut u8
34 }

```

buddy heap 本身也从线性链表改为 AVL tree。旧实现需要遍历整个 size-class 空闲链寻找 buddy；新实现可以按地址直接找到：

```

1 pub struct Heap<const ORDER: usize> {
2     free_list: [avl_tree::AvlTree; ORDER],
3     user: usize,
4     allocated: usize,
5     total: usize,
6 }
7
8 let buddy = current_ptr ^ (1 << current_class);
9 if can_merge && self.free_list[current_class].remove(buddy).is_some()
10 {
11     current_ptr = min(current_ptr, buddy);
12     current_class += 1;
13     continue;
14 }
15 self.free_list[current_class].insert(current_ptr as *mut usize);

```

通过这三个主要的改动，便能够分别解决“频繁获取全局锁”“在锁内逐对象操作”和“单次 buddy 操作线性扫描”三个层次的问题。

3. 实验分析：修改前后的编译时间、加速比等数据对比

3.1 修改前后的编译时间：

最初版本 C: 7961ad6f2dbe5ff2eb87dff1de0e3ab80e370e08，即堆的分配全部要经过一把关中断锁。因为没有跑完，只能截图一个大概的时间。

```
l_idle_pct=8.61 overall_busy_cpus=7.31
BUILDSTORM_GLOBAL phase=arceos_build event=sample_172 online_cpus=8 wall_s=13061.81 capacity_s=104494.51 user_s=4004.44 kernel_s=92127.51 idle_s=8362.56 user_pct=3.83 kernel_pct=88.16 idle_pct=8.00 user_cpus=0.31 kernel_cpus=7.05 busy_cpus=7.36 overall_wall_s=13172.91 overall_user_pct=3.85 overall_kernel_pct=87.57 overall_idle_pct=8.58 overall_busy_cpus=7.31
BUILDSTORM_GLOBAL phase=arceos_build event=sample_173 online_cpus=8 wall_s=13116.73 capacity_s=104933.83 user_s=4023.66 kernel_s=92543.56 idle_s=8366.61 user_pct=3.83 kernel_pct=88.19 idle_pct=7.97 user_cpus=0.31 kernel_cpus=7.06 busy_cpus=7.36 overall_wall_s=13227.83 overall_user_pct=3.85 overall_kernel_pct=87.60 overall_idle_pct=8.54 overall_busy_cpus=7.32
Compiling sbi-rt v0.0.4
BUILDSTORM_GLOBAL phase=arceos_build event=sample_174 online_cpus=8 wall_s=13168.93 capacity_s=105351.44 user_s=4074.05 kernel_s=92904.39 idle_s=8372.99 user_pct=3.87 kernel_pct=88.19 idle_pct=7.95 user_cpus=0.31 kernel_cpus=7.05 busy_cpus=7.36 overall_wall_s=13280.03 overall_user_pct=3.89 overall_kernel_pct=87.60 overall_idle_pct=8.52 overall_busy_cpus=7.32
Compiling aml v0.16.4
BUILDSTORM_GLOBAL phase=arceos_build event=sample_175 online_cpus=8 wall_s=13274.68 capacity_s=106197.41 user_s=4092.30 kernel_s=93723.46 idle_s=8381.65 user_pct=3.85 kernel_pct=88.25 idle_pct=7.89 user_cpus=0.31 kernel_cpus=7.06 busy_cpus=7.37 overall_wall_s=13385.77 overall_user_pct=3.87 overall_kernel_pct=87.67 overall_idle_pct=8.46 overall_busy_cpus=7.32
BUILDSTORM_GLOBAL phase=arceos_build event=sample_176 online_cpus=8 wall_s=13373.10 capacity_s=106984.80 user_s=4105.58 kernel_s=94494.88 idle_s=8384.34 user_pct=3.84 kernel_pct=88.33 idle_pct=7.84 user_cpus=0.31 kernel_cpus=7.07 busy_cpus=7.37 overall_wall_s=13484.20 overall_user_pct=3.86 overall_kernel_pct=87.74 overall_idle_pct=8.40 overall_busy_cpus=7.33
Compiling some-serial v0.7.0 (/work/tgskits/drivers/serial/some-serial)
BUILDSTORM_GLOBAL phase=arceos_build event=sample_177 online_cpus=8 wall_s=13534.98 capacity_s=108279.84 user_s=4132.69 kernel_s=95760.00 idle_s=8387.15 user_pct=3.82 kernel_pct=88.44 idle_pct=7.75 user_cpus=0.31 kernel_cpus=7.08 busy_cpus=7.38 overall_wall_s=13646.08 overall_user_pct=3.84 overall_kernel_pct=87.86 overall_idle_pct=8.30 overall_busy_cpus=7.34
BUILDSTORM_GLOBAL phase=arceos_build event=sample_178 online_cpus=8 wall_s=13774.21 capacity_s=110193.71 user_s=4177.82 kernel_s=97627.04 idle_s=8388.85 user_pct=3.79 kernel_pct=88.60 idle_pct=7.61 user_cpus=0.30 kernel_cpus=7.09 busy_cpus=7.39 overall_wall_s=13885.31 overall_user_pct=3.81 overall_kernel_pct=88.03 overall_idle_pct=8.16 overall_busy_cpus=7.35
BUILDSTORM_GLOBAL phase=arceos_build event=sample_179 online_cpus=8 wall_s=14005.36 capacity_s=112042.88 user_s=4224.64 kernel_s=99428.19 idle_s=8390.04 user_pct=3.77 kernel_pct=88.74 idle_pct=7.49 user_cpus=0.30 kernel_cpus=7.10 busy_cpus=7.40 overall_wall_s=14116.46 overall_user_pct=3.79 overall_kernel_pct=88.18 overall_idle_pct=8.03 overall_busy_cpus=7.36
Compiling kernutil v0.2.2 (/work/tgskits/components/kernutil)
BUILDSTORM_GLOBAL phase=arceos_build event=sample_180 online_cpus=8 wall_s=14256.76 capacity_s=114054.09 user_s=4268.14 kernel_s=101395.35 idle_s=8390.61 user_pct=3.74 kernel_pct=88.90 idle_pct=7.36 user_cpus=0.30 kernel_cpus=7.11 busy_cpus=7.41 overall_wall_s=14367.86 overall_user_pct=3.76 overall_kernel_pct=88.35 overall_idle_pct=7.89 overall_busy_cpus=7.37
BUILDSTORM_GLOBAL phase=arceos_build event=final online_cpus=8 wall_s=14707.60 capacity_s=117660.77 user_s=4291.59 kernel_s=104184.52 idle_s=9184.66 user_pct=3.65 kernel_pct=88.55 idle_pct=7.81 user_cpus=0.29 kernel_cpus=7.08 busy_cpus=7.38 overall_wall_s=14818.70 overall_user_pct=3.67 overall_kernel_pct=88.02 overall_idle_pct=8.32 overall_busy_cpus=7.33
QEMU: Terminated
```

实现第一次优化后 B, 1c0d99aab415ba866cbe9852b9493b70f73cb40f，即实现了 per CPU magazine：

```
Finished "release" profile [optimized] target(s) in 40m 40s
Converting ELF to BIN format...
elf: /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
bin: /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
BUILDSTORM_GLOBAL phase=arceos_build event=sample_71 online_cpus=8 wall_s=2517.08 capacity_s=20136.62 user_s=3695.10 kernel_s=9070.29 idle_s=7362.22 user_pct=18.35 kernel_pct=45.09 idle_pct=36.56 user_cpus=1.47 kernel_cpus=3.61 busy_cpus=5.08 overall_wall_s=2605.21 overall_user_pct=17.96 overall_kernel_pct=44.11 overall_idle_pct=37.93 overall_busy_cpus=4.97 heap_cache_hits=33762143 heap_cache_misses=240778 heap_global_allocs=4075099 heap_global_deallocs=3434111 heap_lock_acquisitions=883217 heap_lock_contented=195413 heap_lock_wait_ns=4117129546160 heap_lock_max_wait_ns=1355220240 heap_contented_alloc=21125 heap_contented_dealloc=22281 heap_contented_refill=70238 heap_contented_drain=81769
BUILDSTORM_GLOBAL phase=arceos_build event=sample_72 online_cpus=8 wall_s=2552.30 capacity_s=20418.37 user_s=3697.45 kernel_s=9160.28 idle_s=7551.65 user_pct=18.11 kernel_pct=44.91 idle_pct=36.98 user_cpus=1.45 kernel_cpus=3.59 busy_cpus=5.04 overall_wall_s=2640.43 overall_user_pct=17.73 overall_kernel_pct=43.95 overall_idle_pct=38.32 overall_busy_cpus=4.93 heap_cache_hits=33859700 heap_cache_misses=241038 heap_global_allocs=4079378 heap_global_deallocs=3455142 heap_lock_acquisitions=885022 heap_lock_contented=196760 heap_lock_wait_ns=4164022397920 heap_lock_max_wait_ns=1355220240 heap_contented_alloc=21206 heap_contented_dealloc=22360 heap_contented_refill=70491 heap_contented_drain=82712
/root/.rustup/toolchains/nightly-2026-05-28-riscv64gc-unknown-linux-gnu/lib/rustlib/riscv64gc-unknown-linux-gnu/bin/llvm-objcopy --strip-all -O binary /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
[axbuild] cargo build package=arceos-helloworld target=scripts/targets/std/pie/riscv64gc-unknown-linux-musl.json config=/work/tgskits/tmp/axbuild/config/arceos-helloworld/build-riscv64gc-unknown-linux-musl-elf.toml ... done (2524.83s)
[axbuild] cargo build elf=/work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
[axbuild] cargo build artifact_dir=/work/tgskits/target/riscv64gc-unknown-linux-musl/release
BUILDSTORM_GLOBAL phase=arceos_build event=final online_cpus=8 wall_s=2574.34 capacity_s=20594.60 user_s=3698.69 kernel_s=9226.54 idle_s=7669.45 user_pct=17.96 kernel_pct=44.80 idle_pct=37.24 user_cpus=1.44 kernel_cpus=3.58 busy_cpus=5.02 overall_wall_s=2662.47 overall_user_pct=17.59 overall_kernel_pct=43.85 overall_idle_pct=38.56 overall_busy_cpus=4.92 heap_cache_hits=33920215 heap_cache_misses=241375 heap_global_allocs=4084915 heap_global_deallocs=3465896 heap_lock_acquisitions=886313 heap_lock_contented=197521 heap_lock_wait_ns=4194268810400 heap_lock_max_wait_ns=1355220240 heap_contented_alloc=21270 heap_contented_dealloc=22417 heap_contented_refill=70689 heap_contented_drain=83145
BUILDSTORM_COMPILE mode=multi ok=true elapsed_s=2584.38 cores=8 bytes=1681000 arch=riscv64
BUILDSTORM_GLOBAL phase=overall event=final online_cpus=8 wall_s=2696.44 capacity_s=21571.51 user_s=3755.70 kernel_s=9401.66 idle_s=8414.15 user_pct=17.41 kernel_pct=43.58 idle_pct=39.01 user_cpus=1.39 kernel_cpus=3.49 busy_cpus=4.88 overall_wall_s=2696.44 overall_user_pct=17.41 overall_kernel_pct=43.58 overall_idle_pct=39.01 overall_busy_cpus=4.88 heap_cache_hits=34452583 heap_cache_misses=241836 heap_global_allocs=4097507 heap_global_deallocs=3479701 heap_lock_acquisitions=897109 heap_lock_contented=197721 heap_lock_wait_ns=4204795097760 heap_lock_max_wait_ns=1355220240 heap_contented_alloc=21309 heap_contented_dealloc=22461 heap_contented_refill=70700 heap_contented_drain=83251
#### OS COMP TEST GROUP END buildstornr ####
```

实现了第二次优化之后，即最终版本 A

```
Converting ELF to BIN format...
elf: /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
bin: /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
/root/.rustup/toolchains/nightly-2026-05-28-riscv64gc-unknown-linux-gnu/lib/rustlib/riscv64gc-unknown-linux-gnu/bin/llvm-objcopy --strip-all -O binary /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld /work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
[axbuild] cargo build package-arceos-helloworld target-scripts/targets/std/pie/riscv64gc-unknown-linux-musl.json config=/work/tgskits/tmp/axbuild/config/arceos-helloworld/build-riscv64gc-unknown-linux-musl-elf.toml ... done (1256.47s)
[axbuild] cargo build elf=/work/tgskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
[axbuild] cargo build artifact_dir=/work/tgskits/target/riscv64gc-unknown-linux-musl/release
BUILDSTORM_GLOBAL phase-arceos_build event-final online_cpus=8 wall_s=1296.02 capacity_s=10368.15 user_s=3963.92 kernel_s=1728.84 idle_s=4675.39 user_pct=38.23 kernel_pct=16.67 idle_pct=45.09 user_cpus=3.06 kernel_cpus=1.33 busy_cpus=4.39 overall_wall_s=1384.89 overall_user_pct=36.23 overall_kernel_pct=16.63 overall_idle_pct=47.14 overall_busy_cpus=4.23 fault_file_shared_pages=541069 fault_file_private_copies=18133 fault_anon_calls=1536664 fault_anon_total_ns=65656397440 fault_anon_total_ns_max=27944800 fault_anon_frame_alloc_ns=12682089040 fault_anon_frame_alloc_ns_max=1556960 fault_anon_zero_ns=21243311120 fault_anon_zero_ns_max=27874560 block_requests=315053 block_completions=315053 block_requested_sectors=3787027 ext4_namespace_acquisitions=7310 ext4_namespace_contentions=625 ext4_namespace_wait_ns=21958601360 ext4_journal_contentions=1182 ext4_inode_contentions=2413 ext4_block_group_acquisitions=674510 ext4_block_group_contentions=18263 ext4_block_group_wait_ns=8836848320 ext4_peak_block_groups=1
BUILDSTORM_COMPILE mode-multi ok=true elapsed_s=1299.99 cores=8 bytes=1681000 arch=riscv64
BUILDSTORM_GLOBAL phase-overall event-final online_cpus=8 wall_s=1401.13 capacity_s=11209.04 user_s=4019.39 kernel_s=1866.66 idle_s=5323.00 user_pct=35.86 kernel_pct=16.65 idle_pct=47.49 user_cpus=2.87 kernel_cpus=1.33 busy_cpus=4.20 overall_wall_s=1401.13 overall_user_pct=35.86 overall_kernel_pct=16.65 overall_idle_pct=47.49 overall_busy_cpus=4.20 fault_file_shared_pages=541410 fault_file_private_copies=18190 fault_anon_calls=1536999 fault_anon_total_ns=65662569680 fault_anon_total_ns_max=27944800 fault_anon_frame_alloc_ns=12683352000 fault_anon_frame_alloc_ns_max=1556960 fault_anon_zero_ns=21243311120 fault_anon_zero_ns_max=27874560 block_requests=316573 block_completions=316573 block_requested_sectors=3807120 ext4_namespace_acquisitions=7311 ext4_namespace_contentions=625 ext4_namespace_wait_ns=21958607120 ext4_journal_contentions=1201 ext4_inode_contentions=2417 ext4_block_group_acquisitions=711485 ext4_block_group_contentions=18308 ext4_block_group_wait_ns=8847495440 ext4_peak_block_groups=1
### OS COMP TEST GROUP END buildstorm ###
[initproc] reaped 3 zombie child(ren) while waiting for /glibc/buildstorm_testcode.sh
[initproc] cleaned /glibc/buildstorm_testcode.sh process_group=2 reaped=0 kill_ret=-3
[initproc] finished /glibc/buildstorm_testcode.sh exit_code=0
[initproc] all official test scripts finished, poweroff
root@ubuntu20:/workspace#
```

指标	A 最新	B `1c0d99aa	C `b7fb26d6
minibuild Cargo	15.92s	15.52s	17.74s
minibuild phase wall	18.85s	18.56s	21.83s
tg-xtask Cargo	37.00s	38.34s	40.55s
tg-xtask phasewall	37.70s	39.23s	42.12s
cargo release 报告	1180s (19m40s)	2440s (40m40s)	未完成
arceos phasewall	1237.95s	2574.34s	≥ 14707.60s
BUILDSTORM_COMPILE elapsed_s	1267.53s	2584.38s	未产生成功结果
buildstorm overall wall	1378.06s	2696.44s	≥ 14818.70s
成功产物	1681000B	1681000B	无完整结果

3.2 加速比

定义：
加速比 S=T 旧/T 新
耗时下降率=(T 旧 - T 新)/T 旧

对比	口径	加速比	耗时下降
A 相对 B	BUILDSTORM_COMPILE	2.039×	50.95%
A 相对 B	arceos phase	2.080×	51.91%
A 相对 B	overall	1.957×	48.89%
B 相对 C	overall	>5.496×	>81.80%
A 相对 C	overall	>10.753×	>90.70%
A 相对 C	arceos phase	>11.881×	>91.58%

因为 C 在日志结束时仍未完成，所以我在这里的处理是通过一个下界来进行划分，`≥` 和 `>` 代表是下界，而不是将 14818.70s 当作完整编译时间。可以清晰的看到，从最初的版本 C 到第一次优化后 B，加速比是大于 5 倍，从 B 到 A，加速比也有两倍。而总的 C 到 A，加速比超过十倍，因此这两次优化是最关键的优化。

4. AI 使用说明与可复现步骤

4.1 AI 使用说明

我们使用 AI 的地方主要是在代码辅助撰写和日志数据统计部分。

在日志代码辅助撰写部分，我们先自己决定需要统计的各项指标，比如用户态、内核态和空闲 CPU 时间、heap 锁获取次数、竞争次数及累计等待时间，然后将指标定义、统计口径、输出格式以及需要插桩的代码路径整理为 prompt，再由 AI 辅助完成性能计数器、日志输出代码和日志解析脚本的初步实现。最后我们会人工复核 AI 所增加的指标是否符合我们的思路，以及日志的输出是否会影响原本脚本的运行以此来决定日志代码是否合理。

在日志数据统计部分，报告中的日志统计主要通过 AI 辅助完成，完成三次实验后，我们将日志输入给 AI，让 AI 抽取不同阶段的不同指标，同时计算加速比等等指标。AI 输出主要用于提高数据提取和计算效率。平时的分析我们也会先让 AI 计算一下异常的部分，然后再我们通过浓缩后的数据来分析瓶颈可能出现在哪里。

在生成代码部分，主要的优化思路是由我们团队提出的，比如第一次优化的 per CPU magement 以及虽然没有实际性作用的批量 drain 和 refill，第二次的 per-CPU slab 方案，都是我们在分析了 AI 给予的日志的总结，然后我们先决定好思路、决定关键数据结构，再将 prompt 输入给 AI，最终生成优化后的代码。然后我们再人工复查这些代码是否合理。

可以发现，我们团队主要是利用 AI 来进行提速，而不是幻想随便给个“帮我优化 buildstorm_testcode.sh”这样的 prompt 来幻想 AI 能够实现所有优化。我们采用以下的协作流程：

- 1 团队定义问题和观测指标
- 2 → AI 辅助实现打点和日志脚本
- 3 → 团队运行实验
- 4 → AI 辅助提取和计算数据
- 5 → 团队分析瓶颈并设计优化方案
- 6 → AI 辅助将方案转化为代码
- 7 → 团队审核代码

4.2 可复现步骤

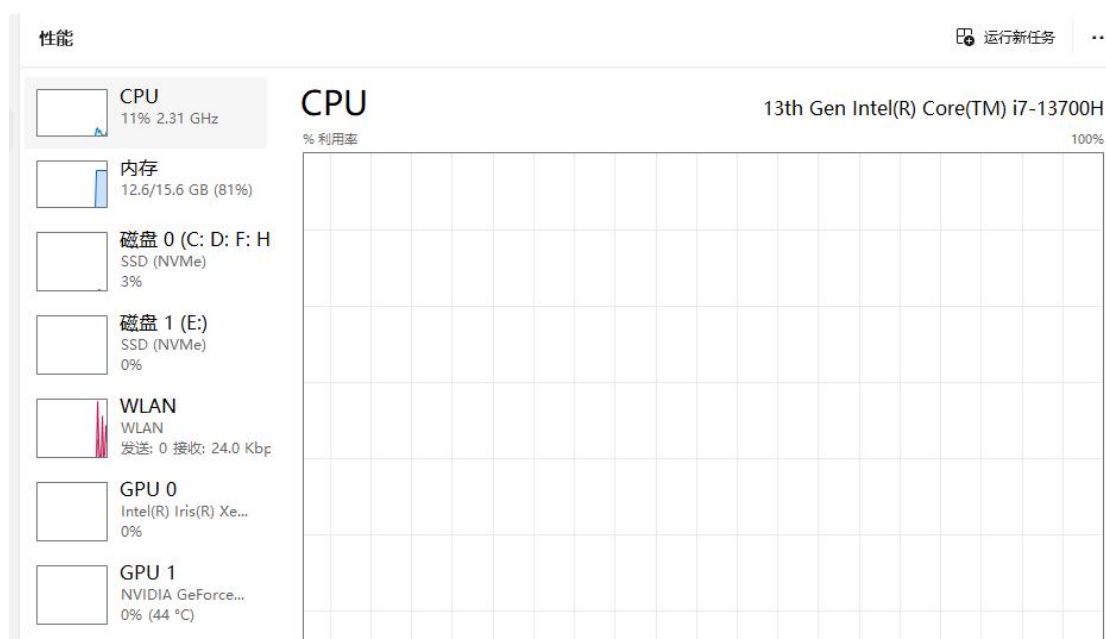
4.2.1 实验环境

为了便于审核和复现实验，我保留了三个版本运行 buildstorm_testcode.sh 时产生的完整日志。日志存放在 Kairix 仓库根目录。

日志文件	版本	对应提交	运行结果
buildstorm-latest.log	A: 最新优化版本	b50a290f9e34d67 0e14191582d1374 0ff92f0016	完成，速度最快
buildstorm-magazine.log	B: per-CPU magazine 版本	1c0d99aab415ba8 66cbe9852b9493b 70f73cb40f	完成，速度居中
buildstorm-global-heap.log	C: 全局 heap 版本	7961ad6f2dbe5ff2 eb87dff1de0e3ab 80e370e08	未完成，长时间缓慢运行

评委老师们可以通过两种方式复核本报告：第一种是直接根据原始日志重新计算结果；第二种是检出对应提交，在相同环境中重新运行实验。

相同环境的话，我的电脑是 14 核 20 线程，具体参数如下



因此按理来说评测机运行的日志会快于我们的日志，这种情况是合理的。

4.2.2 构建与运行

.注：这一章节主要是方便评委老师复现带有日志的 buildstorm_testcode.sh，不带有日志的复现请看 4.2.4

检出需要复现的提交后，将一份未运行过 BuildStorm 的初始 sdcard-rv.img 副本放在 Kairix 仓库根目录，注意要提前有镜像再执行下面的命令，目的是将用户态的程序写入镜像中，然后执行：

```
1 make rkernel_test AUTO_TEST=OFF
```

进入终端后，输入 `buildstorm_2_testcode.sh` 就能够复现带有日志的输出。

4.2.3 本地诊断脚本说明

本文中的性能截图和详细 heap 指标来自仓库中的本地诊断脚本：

```
1 user/buildstorm_1.sh
```

执行 `make all` 时，如果仓库根目录存在 `sdcard-rv.img`，`Makefile` 会在构建阶段挂载该本地镜像，并将 `user/buildstorm_1.sh` 复制为：

```
1 /glibc/buildstorm_2_testcode.sh
```

该脚本保留了主编译阶段的目标和核心命令：

```
1 cargo xtask arceos build \  
2     -p arceos-helloworld \  
3     --arch "$AXARCH"
```

并增加了以下观测功能：

- 1.记录各阶段的开始和结束时间；
- 2.定期只读采样 `/proc/kairix_perf`；
- 3.每隔 30 秒输出 CPU 及内核子系统指标；
- 4.输出 heap 锁竞争、缺页和块设备等诊断信息。

三个提交中的诊断脚本版本随分析过程逐步更新，但主编译目标、核心 Cargo 命令和目标架构清理策略保持一致。诊断脚本只用于分析主编译阶段的性能变化，不会作为正式评分脚本的替代。

4.2.4 使用官方评测流程交叉验证

如果评委老师们希望排除本地诊断脚本带来的影响，可以按照正式评测流程，只提交构建得到的 `kernel-rv`，并使用评测系统提供的原始镜像和官方 `buildstorm_testcode.sh` 进行测试。正式评测日志输出的是：

```
1 TOOLCHAIN_RESULT status=OK  
2 MINIBUILD_RESULT status=OK  
3 BUILDSTORM_BEGIN mode=multi  
4 BUILDSTORM_RESULT mode=multi status=OK ... run=OK
```

其中没有本地诊断脚本特有的：

```
1 BUILDSTORM_DIAG_VERSION  
2 BUILDSTORM_GLOBAL  
3 phase=arceos_build event=sample
```

总的来说，两种复现方式的用途分别为：

- 1 本地诊断流程：
- 2 用于复现详细 CPU、heap 和锁竞争指标
- 3
- 4 官方评测流程：
- 5 用于验证正式编译耗时、产物正确性和最终评分结果